

RUNMAXXIN: Adaptive Music Recommendation for Runners via NLP and Physiological Signal Fusion

Daniele Tegliucci, Alberto Tridente

Bachelor's Degree Programme in Applied Computer Science and Artificial Intelligence (ACSAI)

Course: AI-LAB

Department of Computer Science, Sapienza University of Rome

tegliucci.2169138@studenti.uniroma1.it

tridente.2173368@studenti.uniroma1.it

Abstract

RUNMAXXIN is an edge-compatible system designed to dynamically adapt music to a runner's intent and physiological state. Current solutions rely on static playlists or simplistic heart-rate heuristics that ignore stated workout goals and lack explicit safety constraints. We propose a neuro-symbolic pipeline that processes natural-language prompts through a lightweight SetFit classifier, fuses the extracted intent with physiological effort metrics, and selects music according to BPM, energy, valence, and genre compatibility. A deterministic symbolic layer based on OWL, SPARQL, and SHACL enforces safety and plausibility constraints, overriding the recommender when physiological strain becomes critical. Our evaluation shows that the system improves over keyword-based intent recognition, runs with low latency in end-to-end simulations, and reduces repetitive recommendations through probabilistic sampling.

Keywords: neuro-symbolic AI; natural language processing; sensor fusion; ontology; edge computing; music recommendation.

1 Introduction

Music is widely recognized in sports science as a powerful natural pacer. Through auditory-motor synchronization, an appropriate beats-per-minute (BPM) value can support rhythm, reduce perceived fatigue, and improve the training experience. However, music recommendation for running is not a static task: a runner's needs change according to the workout goal, emotional state, fatigue, speed, cadence, and heart-rate response. The problem addressed by this project is how to build an edge-compatible AI pipeline that connects natural-language intent with physiological signals in order to recommend music that is both motivating and safe.

This problem is relevant in the context of Edge AI, where the system must make decisions with low latency and limited computational resources. Existing solutions often rely either on fixed playlists or on simple heart-rate heuristics. Early systems such as MPTrain [1] explored physiology-based music selection, but they did not provide a flexible natural-language interface. Modern playlist systems can model musical preference, but they usually do not include explicit physiological safety constraints or interpretable symbolic reasoning.

RUNMAXXIN addresses this gap by combining qualitative and quantitative user intent. A prompt such as "I want an easy recovery run" should not be handled like a prompt such as "I want to run at 12 km/h". The first is mainly semantic and mood-oriented, while the second provides a numerical target. The system therefore adapts its internal recommendation weights according to the type of prompt, while also reacting to the current physiological state of the runner.

The contribution of this work is threefold. First, we benchmark a lightweight few-shot SetFit classifier for goal and mood extraction from natural language. Second, we design a physiological controller that maps heart rate, speed, and cadence into an acoustic target vector. Third, we introduce a symbolic

safety layer based on an OWL ontology, SPARQL queries, and SHACL constraints, which validates physiological risk and rejects implausible extracted values.

The remainder of this report is organized as follows. Section 2 reviews related work. Section 3 describes the proposed pipeline and its main components. Section 4 presents the experimental setup, benchmarks, ablation studies, and critical discussion. Section 5 concludes the report and outlines future developments.

2 Related Work

Physiological monitoring for music selection has been explored by systems such as MPTrain [1], which used physiological signals to guide music selection during training. These approaches demonstrated that music can be adapted to physical activity, but they usually required predefined numerical inputs and did not integrate natural-language understanding.

Recent deep learning methods have improved text classification and recommendation, but many transformer-based systems require larger datasets and higher computational resources. In this project, we use SetFit [4], a few-shot learning framework based on Sentence-BERT embeddings [5]. SetFit is suitable for small datasets because it fine-tunes sentence representations through contrastive learning and then trains a lightweight classification head.

Symbolic and ontology-based methods are also relevant for this project. OWL [6], SPARQL [7], and SHACL [8] make it possible to define explicit constraints and query them in an interpretable way. In RUNMAXXIN, the ontology is not used as the main recommender. Instead, it acts as a deterministic safety and validation layer placed around the statistical recommender.

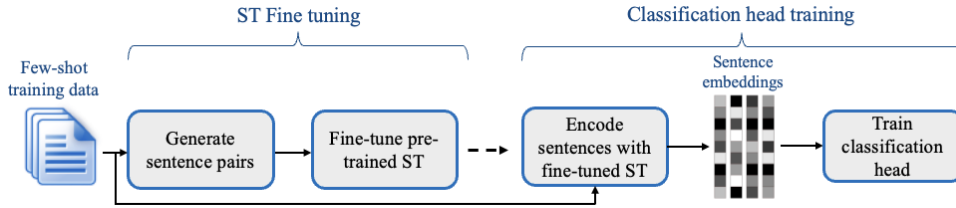


Figure 1: The SetFit training scheme: contrastive fine-tuning of a Sentence Transformer on few-shot sentence pairs.

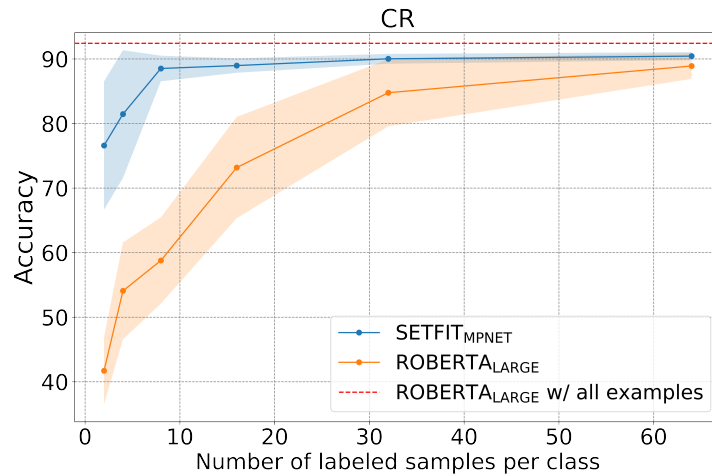


Figure 2: SetFit accuracy in the low-data regime compared to standard fine-tuning.

3 Methodology

The RUNMAXXIN pipeline is a neuro-symbolic system designed for edge-oriented execution. It combines a natural-language module, a physiological analysis module, a controller, a symbolic safety layer, and a music recommender. As shown in Fig. 3, the system transforms user intent and sensor data into a musical target vector, then selects a suitable song from the catalogue.

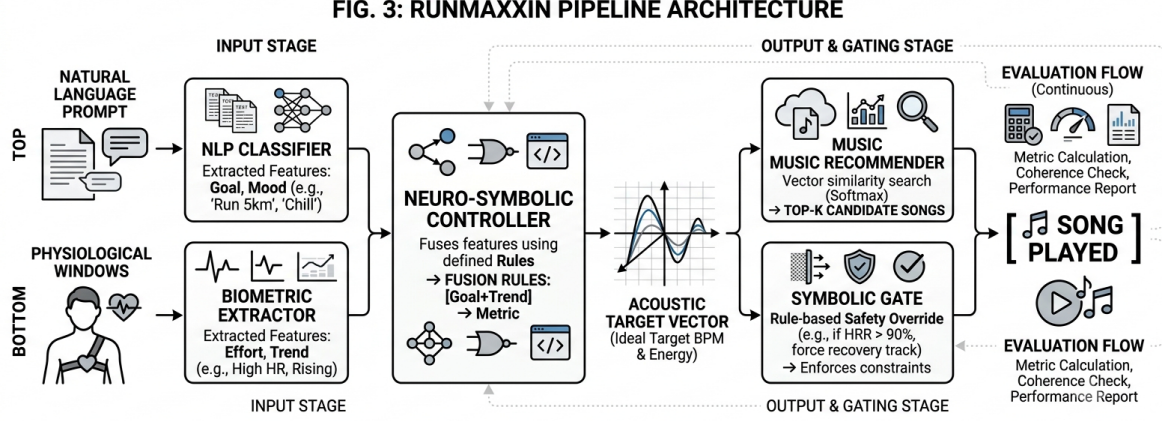


Fig. 3. Detailed RUNMAXXIN pipeline architecture, illustrating dual-path feature extraction (NLP and Biometric), neuro-symbolic fusion, recommendation, symbolic safety gating, and evaluation loop.

Figure 3: RUNMAXXIN pipeline: the system fuses NLP intent with physiological features into an acoustic target vector, which is then filtered through symbolic constraints and passed to the recommender.

3.1 Natural-Language Intent Extraction

The NLP module receives a free-text prompt and extracts the workout goal, the mood, and any numerical information. The goal classifier predicts one of three classes: EasyRun, ModerateRun, and IntenseRun. The mood classifier predicts one of five classes: Neutral, Focused, Energetic, Motivated, and Calm. In addition to classification, the module uses regular expressions to detect speed, pace, distance, and duration. When speed or pace is provided, the system can derive a quantitative target BPM.

The use of SetFit is motivated by the small amount of annotated data. Instead of fine-tuning a large transformer from scratch, SetFit uses a multilingual MiniLM sentence encoder and contrastive sentence pairs, making it more appropriate for a few-shot setting.

3.2 Physiological Mapping and Signal Fusion

The physiological module processes heart rate, speed, and cadence. Effort is normalized using Heart Rate Reserve (HRR), following the Karvonen formula [2]:

$$HRR = \frac{HR_{current} - HR_{rest}}{HR_{max} - HR_{rest}}.$$

The HRR value is then mapped to effort classes: LowEffort, TargetEffort, HighEffort, and VeryHighEffort. The system also estimates trend information by computing the slope of the heart-rate signal over a short temporal window.

To align music with running rhythm, the system maps speed to a target cadence-inspired BPM using the following heuristic:

$$BPM = clamp(134 + 2.9 \cdot speed_{kmh}, 150, 190).$$

Mood is mapped to valence according to a valence-arousal interpretation inspired by Russell’s circumplex model of affect [3]. For example, Calm corresponds to lower valence/arousal targets, while Energetic and Motivated correspond to higher values.

3.3 Controller

The controller combines the NLP output and physiological analysis. If the prompt is quantitative, the system prioritizes BPM matching and uses a narrow BPM tolerance. If the prompt is qualitative, the system gives more importance to mood-compatible genres, energy, and valence. The controller also handles warm-up behaviour, effort adaptation, previous-song BPM variation, and recovery mode.

The final output of the controller is a target object containing BPM, energy, valence, feature weights, BPM tolerance, softmax temperature, allowed genres, effort band, and regime. This target is then passed to the recommender.

3.4 Edge-Optimized Recommendation

The recommender represents each track as a vector:

$$\mathbf{v}_i = [bpm_i, energy_i, valence_i].$$

It compares each song to the controller target using a weighted Euclidean distance:

$$d_i = \sqrt{w_{bpm}(\Delta bpm_i)^2 + w_{energy}(\Delta energy_i)^2 + w_{valence}(\Delta valence_i)^2}.$$

Distances are converted into probabilities through a softmax function with temperature τ :

$$P_i = \frac{\exp(-d_i/\tau)}{\sum_{j \in \mathbf{S}} \exp(-d_j/\tau)}.$$

This allows the system to sample from a candidate pool rather than always selecting the nearest track. The goal is to preserve target compatibility while reducing playlist repetition.

3.5 Symbolic Safety Gating

The symbolic layer defines hard constraints that should not be left to the statistical recommender. The critical HRR threshold is stored in the ontology and checked through a SPARQL query. If HRR becomes critical, the controller activates recovery mode and shifts the target toward lower BPM and lower energy.

The symbolic module also validates extracted speed values through SHACL. This prevents unrealistic values, such as negative speeds or physiologically impossible speeds, from being passed to the BPM mapping function. Finally, the effort gate checks whether the effort labels associated with a candidate song are compatible with the effort band allowed by the controller.

4 Experimental Setup

4.1 Dataset

The experimental evaluation combines three types of data: manually annotated natural-language prompts, a Spotify-based music catalogue, and simulated physiological running sessions. The natural-language data were hand-crafted by the authors and split into two few-shot classification datasets, one for workout-goal recognition and one for mood recognition. The goal dataset contains 82 sentences divided into three classes: EasyRun, ModerateRun, and IntenseRun. It is split into 46 training examples and 36 held-out test examples, with 12 test sentences per class. The mood dataset contains 95 sentences divided into five classes: Neutral, Focused, Energetic, Motivated, and Calm. It is split into 55 training examples and 40 held-out test examples, with 8 test sentences per class. No validation set is used, following a few-shot protocol.

The music catalogue contains 89,503 Spotify tracks represented through metadata and audio features. The recommender uses track identifier, title, artist, genre, BPM, energy, and valence. Additional fields, such as Spotify URL, mood support, goal support, and effort compatibility, are used when available for output formatting and symbolic filtering. The catalogue is used as provided, except for basic cleaning

steps performed at loading time: missing essential fields are discarded, numerical columns are converted to numeric values, invalid BPM or out-of-range energy/valence values are removed, and duplicate song identifiers are dropped.

Finally, six simulated physiological sessions are used to evaluate the adaptive part of the system. Each session represents a different running behaviour: steady pacing, push-then-fatigue, negative split, intervals, easy recovery, and beginner struggle. These sessions are not intended to replace real smartwatch recordings; rather, they provide controlled and reproducible physiological conditions for testing safety, adaptation, and recommendation behaviour.

Table 1: Dataset summary.

Dataset	Classes	Train	Val.	Test / Size	Input format
Goal intent	3	46	0	36	free text
Mood intent	5	55	0	40	free text
Spotify catalogue	–	–	–	89,503	audio features + metadata
Physiological sessions	6 archetypes	–	–	6 sessions	HR, speed, cadence
End-to-end simulations	–	–	–	90 runs	prompt + simulated physiology

The main limitation of the NLP datasets is their small size and manual construction, which may introduce lexical bias. The Spotify catalogue also contains duplicates and near-duplicates, which can reduce perceived diversity. The physiological sessions are simulated and therefore cannot fully capture real-world effects such as sensor noise, terrain changes, fatigue drift, or individual variability in heart-rate response.

4.2 Training and Evaluation Protocol

The NLP classifiers are implemented using SetFit with the `paraphrase-multilingual-MiniLM` family of sentence encoders, specifically the `MiniLM-L12-v2` variant. Two independent classifiers are trained: one for workout-goal recognition and one for mood recognition. The encoder is contrastively fine-tuned on sentence pairs using cosine-similarity loss, followed by a logistic-regression classification head. Training is performed for one epoch with batch size 16 on a MacBook M2. We do not perform hyperparameter tuning, because the purpose of the experiment is to evaluate a lightweight few-shot setup rather than maximize performance through extensive search.

We compare the goal classifier against a deterministic keyword-based rule baseline. This baseline reflects the simplest deployable alternative: it checks for manually defined lexical triggers such as “ripetute”, “sprint”, “recupero”, or “ritmo costante”. For mood recognition, we report SetFit performance directly.

The controller, symbolic layer, and recommender do not require supervised training. They are evaluated through simulation and ablation. The end-to-end tester executes the full system from user prompt to playlist generation, combining NLP intent extraction, physiological signal processing, controller decision, symbolic validation, and music recommendation.

Table 2: Training configuration for the NLP classifiers.

Parameter	Value
Framework	SetFit / Sentence-Transformers
Encoder	paraphrase-multilingual-MiniLM-L12-v2
Classification head	Logistic regression
Optimizer	AdamW / SetFit default
Batch size	16
Epochs	1
Loss function	Cosine-similarity contrastive loss
Main metrics	Accuracy, Macro-F1, latency
Hardware	MacBook M2

For end-to-end experiments, we evaluate the system on 90 simulated runs obtained from 3 prompts, 6 physiological archetypes, and 5 random seeds. The three prompts cover an intense quantitative request, a moderate qualitative request, and an easy recovery request. The random seed controls physiological noise and recommendation sampling, making every run reproducible. We report both predictive metrics and system-level metrics: NLP accuracy and macro-F1, symbolic validation correctness, recovery activations, average and peak HRR, recommendation diversity, repeat rate, target distance, BPM error, and end-to-end latency.

4.3 Results

4.3.1 NLP Classification

On goal classification, SetFit achieves 0.778 accuracy and 0.771 macro-F1, outperforming the keyword baseline, which reaches 0.556 accuracy and 0.534 macro-F1. This corresponds to a 22-point gain in accuracy, showing that the learned sentence representation generalizes beyond the explicit lexical triggers encoded in the baseline. The mood classifier reaches 0.800 accuracy and 0.810 macro-F1, indicating that the learned representation also captures affective information from short user prompts.

Table 3: NLP classification results.

Method	Accuracy	Macro-F1	Latency
Keyword baseline (goal)	0.556	0.534	< 1 ms
SetFit (goal)	0.778	0.771	16 ms
SetFit (mood)	0.800	0.810	16 ms

The improvement is not uniform across classes. As shown in Fig. 5, ModerateRun and EasyRun obtain robust F1-scores, while IntenseRun remains the weakest class. The confusion matrix in Fig. 6 shows that intense prompts without explicit sports vocabulary are sometimes misclassified as EasyRun. This suggests that the model still depends on a limited set of lexical patterns learned from the few-shot dataset. In the mood task, Fig. 7 shows a tendency to collapse uncertain or ambiguous prompts into Neutral, reducing recall for Motivated and Focused.

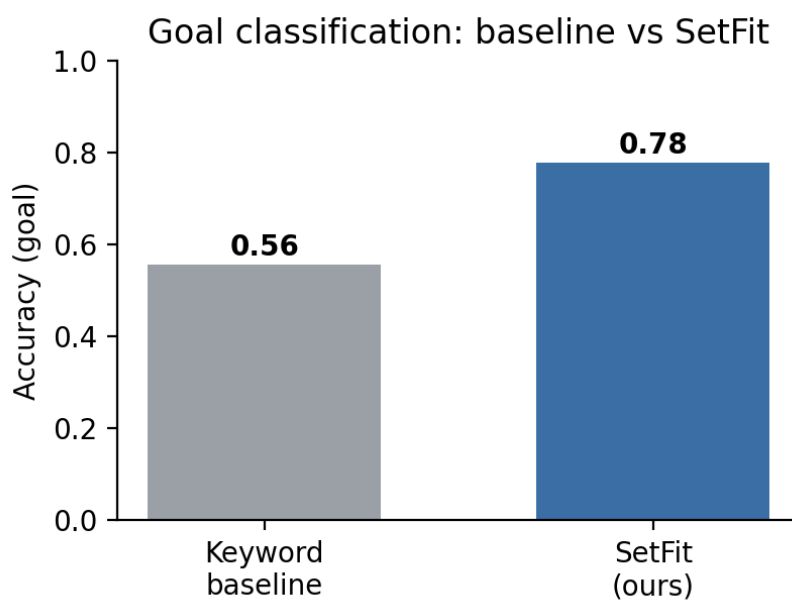


Figure 4: Goal classification accuracy: keyword baseline vs. SetFit.

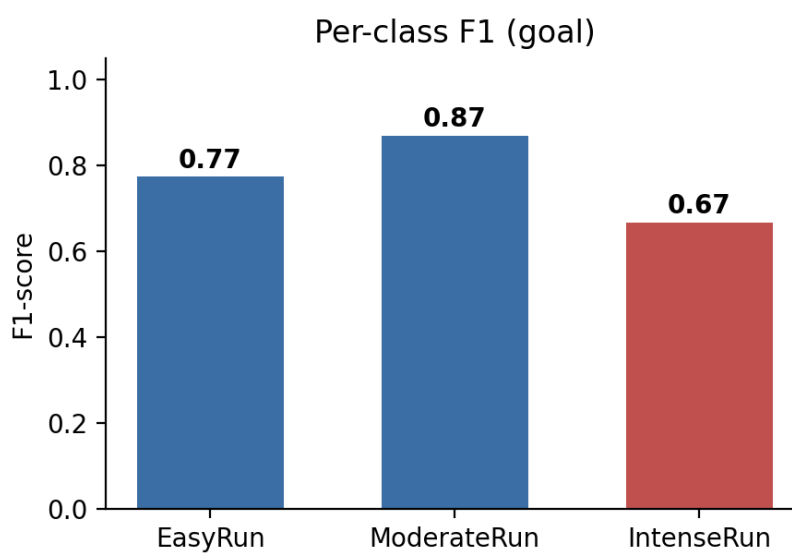


Figure 5: Per-class F1 for goal classification. IntenseRun is the weakest class, mainly due to low recall.

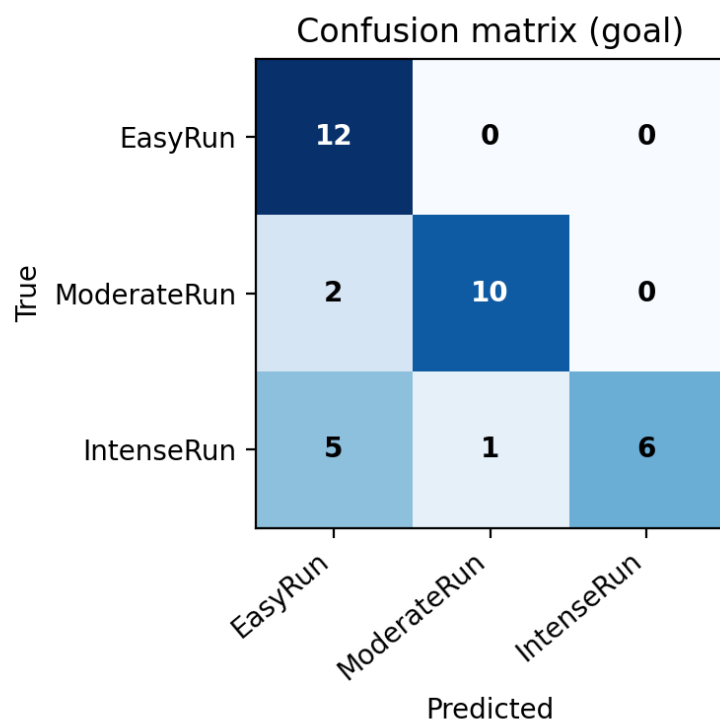


Figure 6: Goal confusion matrix: intense prompts without explicit race vocabulary are sometimes mis-read as EasyRun.

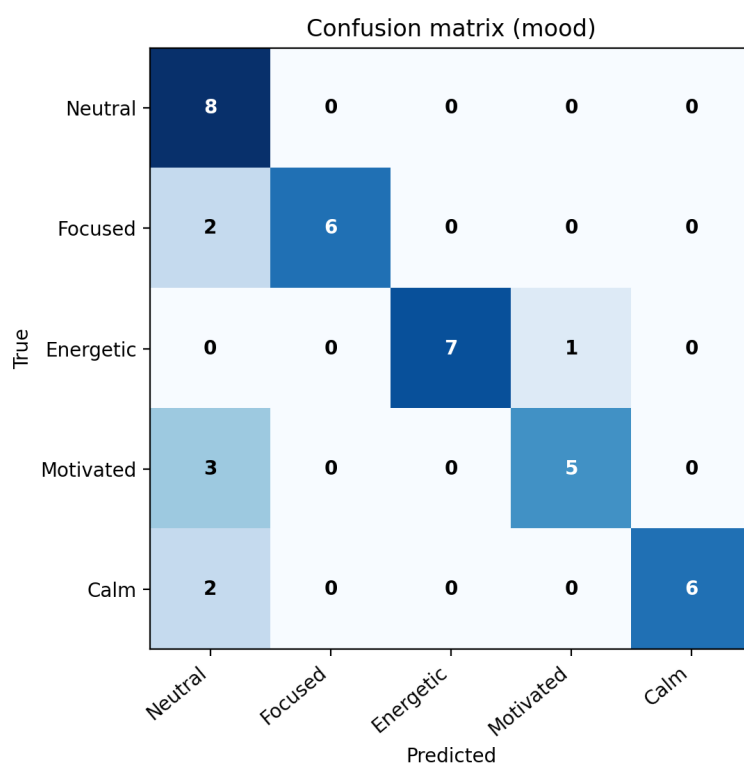


Figure 7: Mood confusion matrix: uncertain prompts tend to fall back onto Neutral.

4.3.2 End-to-End Simulation

We also evaluate RUNMAXXIN as a complete adaptive system. Each test run combines a prompt, a physiological archetype, and a random seed. The generated BPM and speed values are processed by the physiological module, converted into HRR, effort state, and trend state, then passed to the controller. The controller produces a target vector containing BPM, energy, valence, weights, tolerance, temperature, allowed genres, and effort band. Finally, the recommender selects songs from the catalogue using weighted distance, softmax probabilities, and probabilistic sampling.

Table 4: End-to-end simulation metrics over 90 runs.

Metric	Value
Number of prompts	3
Number of physiological archetypes	6
Number of seeds per condition	5
Total simulated runs	90
Mean songs per run	3.01
Mean HRR / peak HRR	0.643 / 0.761
Mean speed / peak speed	11.49 / 13.49 km/h
Recovery activations per run	0.11
Warm-up songs per run	2.00
Quantitative songs per run	0.30
Qualitative songs per run	0.60
Mean target distance	0.061
Mean BPM error	0.36 BPM
Mean end-to-end latency per decision	56.85 ms

The end-to-end results show that the system operates in real time under the simulated workload. The average decision latency is 56.85 ms, dominated by the recommender stage. This is expected, because the recommender compares the target against a catalogue of 89,503 tracks. The controller adds an average of 37.17 ms, mainly due to the symbolic safety check on HRR. Since the simulated sessions last only eight minutes, the first two songs are usually selected during warm-up, which explains the average of 2.00 warm-up songs per run.

Table 5: Average module latency per recommendation decision.

Component	Mean latency
Controller	37.17 ms
Recommender	19.56 ms
Effort gate and final selection	0.12 ms
Full decision step	56.85 ms

The physiological archetypes produce different system behaviours. The interval archetype triggers the largest number of recovery decisions, because peak HRR reaches 0.904 on average. The push-fatigue archetype reaches the highest mean HRR, while easy recovery remains clearly below the critical region.

Table 6: End-to-end behaviour by physiological archetype.

Archetype	Mean HRR	Peak HRR	Recovery/run	Mean speed
steady	0.595	0.663	0.00	12.13 km/h
push_fatigue	0.767	0.860	0.07	14.49 km/h
negative_split	0.701	0.834	0.00	12.58 km/h
intervals	0.672	0.904	0.60	11.85 km/h
easy_recovery	0.475	0.519	0.00	9.90 km/h
beginner_struggle	0.645	0.788	0.00	7.98 km/h

4.3.3 Symbolic Validation

The symbolic layer is evaluated through three unit-level checks. The HRR safety query correctly classifies all tested HRR cases around the critical threshold. The speed constraint accepts plausible extracted speeds and rejects implausible values outside the interval $(0, 45]$ km/h. The effort compatibility query correctly detects whether the effort labels associated with a song intersect the effort band allowed by the target.

Table 7: Symbolic validation results.

Symbolic check	Accuracy	Mean latency
Critical HRR detection via SPARQL	1.000	33.62 ms
Speed constraint validation $(0, 45]$	1.000	—
Effort compatibility via SPARQL	1.000	20.17 ms

The symbolic speed gate is reported as a correctness test rather than as a latency test, because its execution depends on the local `pyshacl` runtime. In contrast, the SPARQL-based HRR and effort checks are directly measured.

4.3.4 Ablation Studies

We evaluate three ablations. First, we compare deterministic Top-K recommendation with probabilistic sampling. The deterministic version always returns the closest tracks, while the probabilistic version samples from a candidate pool using softmax probabilities. Second, we compare recommendation before and after the symbolic effort gate. Third, we compare system behaviour across different prompt regimes and physiological archetypes.

Table 8: Recommender diversity ablation over 100 repeated runs.

Method	Unique songs	Repeat rate	Mean distance@3
Deterministic Top-K	3	0.990	0.0216
Probabilistic sampling	20	0.933	0.0331

The deterministic recommender obtains the lowest distance, but it repeatedly selects the same three tracks. Probabilistic sampling increases the number of unique songs from 3 to 20 across 100 repeated runs. This confirms that the sampling modification introduces meaningful diversity, at the cost of a small increase in target distance.

Table 9: Effect of the symbolic effort gate in end-to-end simulations.

Metric	Value
Candidate effort-violation rate before gate	0.101
Accepted effort-violation rate after gate	0.067
Fallback-to-ungated rate	0.067
Exploration rate after sampling	0.804

The effort gate reduces effort-incompatible choices, but it does not eliminate them completely. The remaining 6.7% accepted violation rate corresponds to fallback cases in which no compatible track is available in the sampled candidate pool. This is an important limitation: the symbolic gate improves safety and coherence, but its effectiveness depends on the presence of effort-compatible tracks in the candidate set.

Table 10: Behaviour by prompt regime.

Prompt type	Recovery/run	Quantitative/run	Qualitative/run	Mean BPM error
Intense quantitative	0.10	0.90	0.00	0.11
Moderate qualitative	0.13	0.00	0.87	0.42
Easy recovery	0.10	0.00	0.93	0.56

The quantitative prompt produces the lowest BPM error, because it forces the controller to search around a narrow target BPM. The qualitative prompts produce larger BPM error because they follow the current speed and physiological state more flexibly, with wider tolerance and stronger influence from mood-compatible genres.

4.4 Critical Discussion

The results show that the learned NLP component is useful but not sufficient to explain the full behaviour of RUNMAXXIN. SetFit clearly outperforms the keyword baseline on goal classification, with a 22-point accuracy improvement. This indicates that sentence-level representations capture variations that handcrafted keywords miss. However, the error analysis also reveals the weakness of the few-shot setting: IntenseRun is the most fragile class, especially when the prompt expresses intensity without technical sports vocabulary. This is coherent with the expected behaviour of few-shot models: they can generalize better than rules, but their robustness is still limited by the lexical diversity of the training data.

The comparison with larger transformer models must be interpreted carefully. We did not fine-tune RoBERTa or similar architectures, so we do not claim that SetFit is superior to them. Rather, SetFit was chosen because it is appropriate for a small hand-crafted dataset and offers low inference latency. Larger models may generalize better when more annotated data are available, but they would also require a more expensive training and validation protocol. In this project, the relevant comparison is therefore not SetFit against large transformer fine-tuning, but few-shot SetFit against deterministic rules under limited annotation budget.

The end-to-end evaluation is more representative of the actual system than the NLP benchmark alone. RUNMAXXIN is designed as an adaptive pipeline: the final recommendation depends not only on the initial prompt, but also on HRR, effort state, trend, warm-up phase, previous song BPM, symbolic safety checks, and probabilistic sampling. Therefore, high NLP accuracy does not automatically imply good playlist behaviour.

The symbolic layer is important for safety and interpretability. The HRR threshold is queried from the ontology, while implausible speeds extracted from text can be rejected through SHACL validation. However, the symbolic gate is only as good as the labels available in the catalogue. If a track has missing or noisy effort annotations, the gate may either reject suitable songs or fall back to ungated candidates. This explains the remaining 6.7% accepted effort-violation rate observed in the end-to-end simulations.

The recommender ablation confirms a clear trade-off between precision and diversity. Deterministic Top-K produces the lowest target distance, but it repeatedly returns the same tracks. Probabilistic sam-

pling increases musical diversity substantially, raising the number of unique tracks from 3 to 20 over 100 repeated runs, while keeping the mean distance low.

The main limitations are fourfold. First, the NLP datasets are small and manually written by the authors, which may introduce lexical and stylistic bias. Second, the physiological sessions are simulated, so they do not capture the full complexity of real smartwatch data, sensor drift, terrain changes, or individual cardiovascular differences. Third, the Spotify catalogue contains duplicates and near-duplicates, which can reduce perceived diversity even when song identifiers differ. Fourth, the current recommender is content-based and does not learn from user feedback over time.

5 Conclusion

RUNMAXXIN addresses the problem of making music recommendation for running more adaptive, safer, and more aware of the user’s real condition. Instead of relying on a static playlist, the system combines natural-language intent, physiological signals, symbolic constraints, and music features to select tracks that better fit the current phase of the workout.

The main strength of the model is its modularity. The SetFit component provides a lightweight NLP layer suitable for few-shot learning, the physiological module captures the runner’s effort through HRR, the controller adapts the acoustic target in real time, and the symbolic layer adds interpretable safety rules. The experiments confirm this design choice: SetFit improves over the keyword baseline, the end-to-end pipeline runs with low latency, and probabilistic sampling reduces repetitive recommendations while keeping the selected songs close to the target.

At the same time, the system has clear limitations. The NLP datasets are small and manually written, so the model may still fail on unusual prompts or expressions not represented in the training set. The physiological data are simulated, which makes the evaluation reproducible but less realistic than tests with real wearable sensors. Moreover, the recommender is still content-based and does not yet learn from long-term user preferences.

Future work should focus on three directions. First, the recommender should be connected to a larger and more dynamic Spotify catalogue, making the full set of available tracks usable rather than relying only on a local snapshot. Second, the pipeline should be tested with real smartwatch data to evaluate its behaviour under real fatigue, sensor noise, and different running conditions. Third, the same architecture could be extended beyond running to other sports, such as cycling, gym training, skiing, or walking. A further direction is the integration of computer vision, for example to estimate posture, movement quality, fatigue, or exercise type from visual input. In this way, RUNMAXXIN could evolve from a running music recommender into a broader adaptive training assistant.

Acknowledgments

We acknowledge the open-source Python ecosystem used in this project, including libraries such as NumPy, pandas, Sentence-Transformers, SetFit, RDFLib, pySHACL, and Rich. We also acknowledge the Spotify-based music catalogue and the simulated physiological data used to build and evaluate the prototype.

References

- [1] N. Oliver and F. Flores-Mangas. *MPTrain: A Mobile, Music and Physiology-Based Personal Trainer*. Proceedings of the 8th Conference on Human-Computer Interaction with Mobile Devices and Services, 2006.
- [2] M. J. Karvonen, E. Kentala, and O. Mustala. *The Effects of Training on Heart Rate: A Longitudinal Study*. *Annales Medicinæ Experimentalis et Biologiæ Fenniae*, 35(3):307–315, 1957.

- [3] J. A. Russell. *A Circumplex Model of Affect*. Journal of Personality and Social Psychology, 39(6):1161–1178, 1980.
- [4] L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat, and O. Pereg. *Efficient Few-Shot Learning Without Prompts*. arXiv preprint arXiv:2209.11055, 2022.
- [5] N. Reimers and I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP-IJCNLP, 2019.
- [6] D. L. McGuinness and F. van Harmelen. *OWL Web Ontology Language Overview*. World Wide Web Consortium, 2004.
- [7] S. Harris and A. Seaborne. *SPARQL 1.1 Query Language*. World Wide Web Consortium, 2013.
- [8] H. Knublauch and D. Kontokostas. *Shapes Constraint Language (SHACL)*. World Wide Web Consortium, 2017.
- [9] RDFLib Team. *RDFLib: A Python Library for Working with RDF*. Available at: <https://rdflib.readthedocs.io/>
- [10] RDFLib Team. *pySHACL: A Python SHACL Validator*. Available at: <https://github.com/RDFLib/pySHACL>